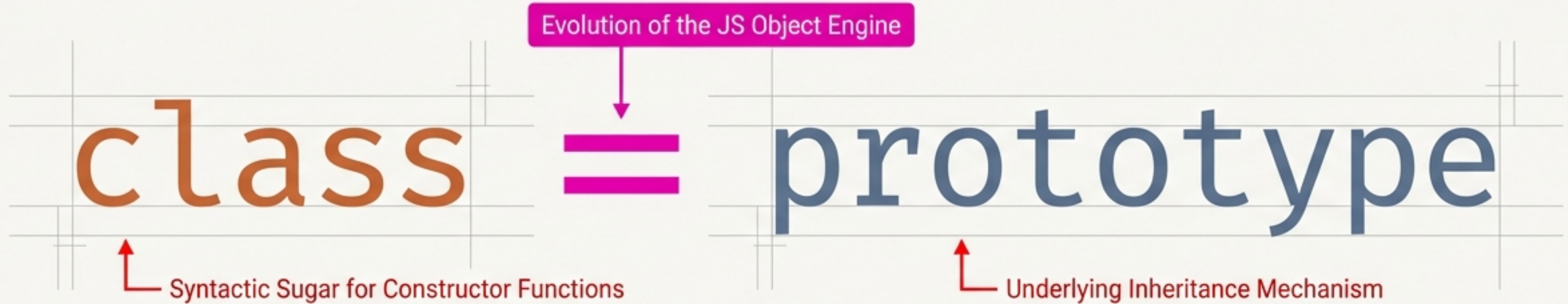


Demystifying JavaScript Classes

Unmasking Syntactic Sugar: Prototypes, Privacy, and the Evolution of the JS Object Engine.



Modern Syntax: The 'class' Keyword

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
  sayHello() {  
    return `Hello, my name is ${this.name}`;  
  }  
}  
  
const user1 = new User('Alice');
```

← Blueprint Definition

← Instance Creation

- ✓ **Clean & Declarative:** Intuitive structure, familiar to OOP.
- ✓ **Encapsulation Support:** Private fields (#) and methods available.

The Engine Underneath: Constructor Functions & Prototypes

```
function User(name) {  
  this.name = name;  
}  
  
User.prototype.sayHello = function() {  
  return `Hello, my name is ${this.name}`;  
};  
  
const user1 = new User('Bob');
```

← Constructor Function

← Shared Behavior

- ✓ **Functional & Flexible:** Direct access to the prototype chain.
- ✗ **More Verbose & Implicit:** Can lead to prototype pollution if not careful.

What is a JavaScript Class?

```
class User {  
  constructor(name) {}  
  sayHi() {}  
}
```

Creates a special function named User (built from the constructor).

Stores all methods (like sayHi) inside User.prototype.

```
typeof User === 'function'
```


The Class Blueprint

Declaration.
Subject to **Temporal
Dead Zone** (not
hoisted like
function
declarations).

Fires automatically
on **new**. Initializes
object state.

```
class MyClass {  
  constructor(name) {  
    this.name = name;  
  }  
  sayHi() {  
    alert(this.name);  
  }  
}  
  
let user = new MyClass('John');
```

⚠ CAUTION: No
commas allowed
between class
methods!
(Causes a
SyntaxError).

How Classes Differ from Plain Functions

	<code>class User</code>	<code>function User()</code>
Invocation	 Must use new (Throws error otherwise due to internal <code>[[IsClassConstructor]]</code> flag).	 Can be called without new.
Enumerability	 Methods are non-enumerable (for..in skips them).	 Methods on prototype are enumerable by default.
Strictness	 Implicitly wrapped in 'use strict'.	 Inherits global strictness.

Class Fields & Context Binding

❌ The Problem: Losing Context

```
let button = new Button('Submit');  
setTimeout(button.click, 1000);
```



Result: undefined. Standard methods lose their **'this' context** when passed as callbacks.

✅ The Solution: Class Fields

```
class Button {  
  constructor(value) {  
    this.value = value;  
  }  
  click = () => {  
    alert(this.value);  
  }  
}
```

Class fields create a separate **bound** function for each object instance. **'this'** is **permanently locked to the object**.

Smart Properties: get and set

Data Validation

```
set name(value) {  
  if (value.length < 4) {  
    alert('Name is too short');  
    return;  
  }  
  this._name = value;  
}
```

Wraps internal properties to enforce rules seamlessly.



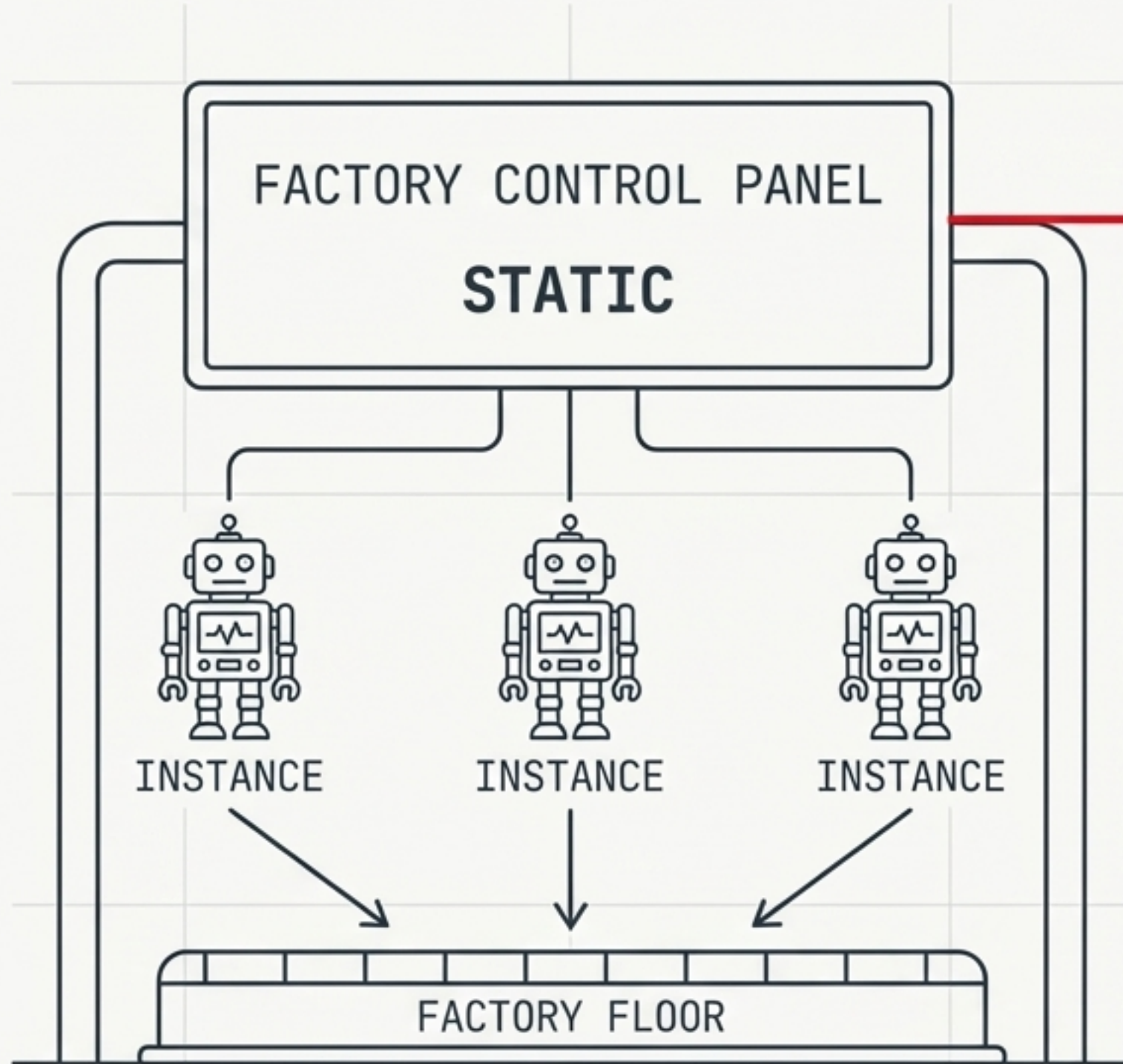
Backwards Compatibility

```
get age() {  
  let year = new Date().getFullYear();  
  return year - this.birthdate.getFullYear();  
}
```

Swaps static data for dynamic logic without breaking existing code that calls 'user.age'.



static: Belonging to the Blueprint



```
class Article {  
    static compare(articleA, articleB) {  
        return articleA.date - articleB.date;  
    }  
  
    read() {  
        console.log(this.content);  
    }  
}
```

Attached to the Class itself
(Article.compare), not the prototype.

Ideal for utility functions, fixed
configurations, or instance cloning/creation.

True Privacy with the # Prefix

- The # prefix enforces hard privacy at the language level.

```
class CoffeeMachine {  
  #waterLimit = 200;  
  
  #checkWater() {  
    return this.#waterLimit;  
  }  
}
```

```
let machine = new CoffeeMachine();  
machine.#waterLimit = 1000;
```

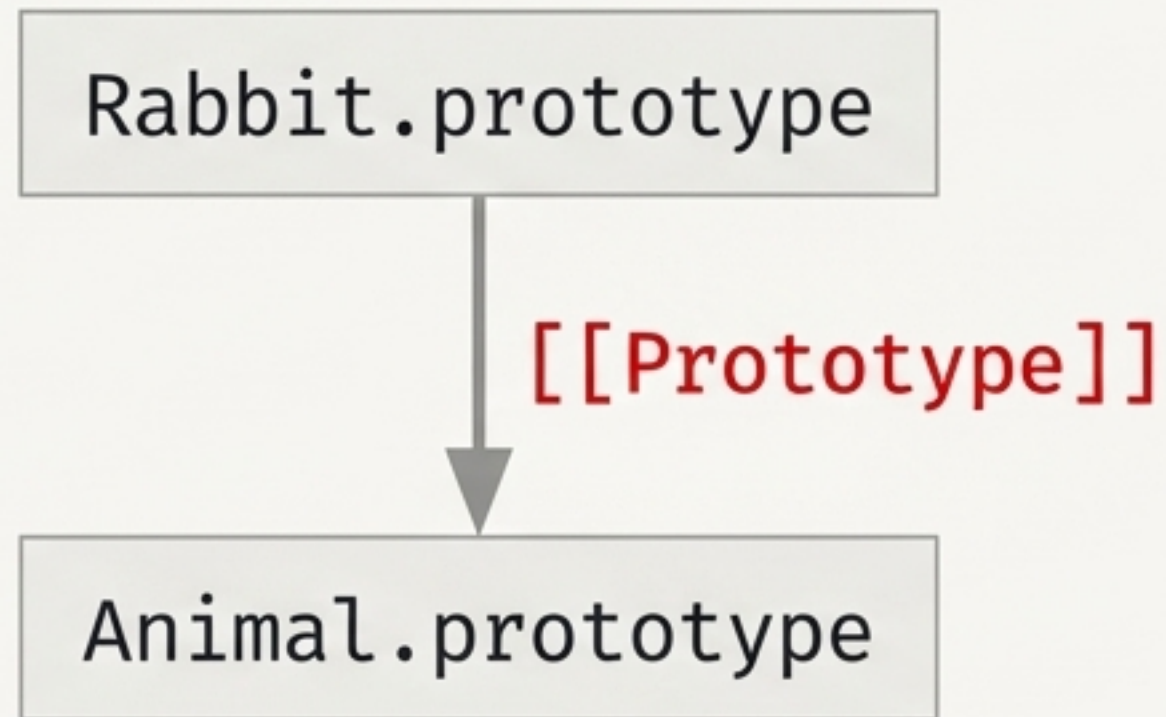
ACCESS DENIED - SyntaxError

- Cannot be read, written, or proxied from outside the class body.

- Use the 'in' operator inside the class to check for private field existence.

Building on Foundations: extends

```
class Rabbit extends Animal { ... }
```



`extends` does two things: sets the prototype for the class constructor, and links the instances' prototypes, granting access to inherited methods.

Calling the Parent with super

```
class Rabbit extends Animal {  
  constructor(name, earLength) {  
    super(name);  
    this.earLength = earLength;  
  }  
}
```

**ReferenceError if super()
is not called first!**

Engine Rule: In a derived class, the constructor has a special internal label. It expects the parent constructor to create the 'this' object. You must call super() before accessing 'this'.

How Each Pattern Fixed the Last

Global
Functions

Flaw: Global scope pollution.

IIFEs

Flaw: Cannot be instantiated (one-off).

Fixes scope.

Function Factories
/ Closures

Flaw: Massive Memory Bloat (methods duplicated on every single instance).

Fixes instantiation
& provides privacy.

Prototypes

Flaw: Clunky syntax, split across multiple code blocks.
No hard privacy.

Fixes memory
(shared methods).

Classes + # Fields
Fixes syntax & privacy.

Provides memory efficiency, clean grouping, and true encapsulation.

The JS Architecture Matrix

	Syntax Cleanliness	Memory Efficiency	Hard Privacy	Context ('this')
Closures / Factories	✓ Clean	✗ Bloated (Duplicated)	✓ Lexical Scoping	✓ Locked
Prototypes (ES5)	✗ Messy / Split	✓ Shared	✗ Honor System (⌋)	✗ Dynamic / Lost easily
Modern Classes (ES6+)	✓ Unified	✓ Shared	✓ # Private Fields	⚠ Fixable via Class Fields

Modern Classes finally combine the memory efficiency of Prototypes with the privacy and clean syntax of Factories.

The Ultimate Class Cheatsheet

Inheritance

Static Property

Private Field

```
class SecureServer extends Server {  
  static maxConnections = 100;  
  #apiKey;
```

```
  constructor(url, key) {  
    super(url);  
    this.#apiKey = key;  
  }
```

Super & Init

Accessor

```
  get keyStatus() {  
    return this.#apiKey ? 'Active' : 'Missing';  
  }
```

Bound Class Field

```
  connect = () => {  
    console.log(`Connecting to ${this.url}`);  
  }  
}
```

The Blueprint Mastered: Syntactic sugar on the surface, a finely-tuned prototype engine underneath.